Нечёткая когнитивная модель

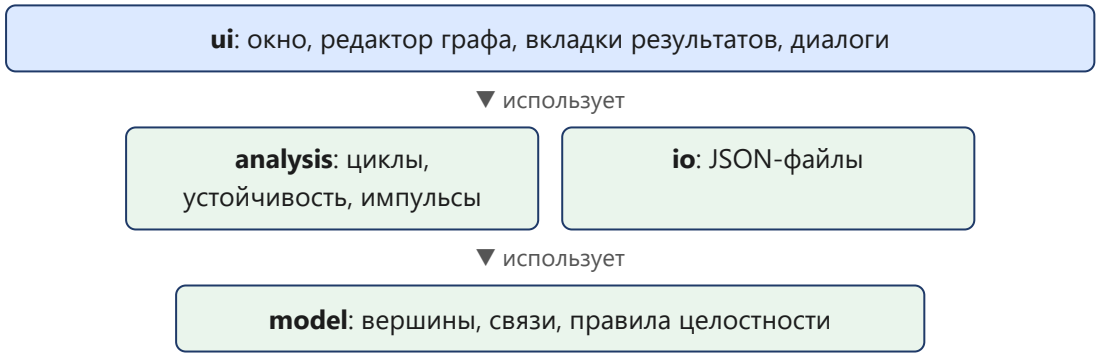
Пояснение к защите: архитектура, классы и формулы · Java 21, Swing, Maven

1. Что делает программа

- **Редактор графа.** Вершины (код C1, C2… назначается автоматически, плюс название) и направленные связи с весом из [−1; 1]. Положительные связи нарисованы зелёной сплошной линией, отрицательные красным пунктиром; есть петли и встречные связи; вершины и связи можно двигать мышью. Есть отмена и повтор, масштаб, автоматическая расстановка.
- **Вкладка «Анализ».** Все циклы модели и знак каждого, вывод о структурной устойчивости; собственные числа матрицы отношений и вывод об устойчивости по возмущению и по начальному значению.
- **Вкладка «Импульсы».** План эксперимента и 6 графиков импульсных процессов: 3 случайные вершины, в каждую импульс +1 и −1.
- **Файл.** Сохранение и загрузка модели в JSON с проверкой данных.

2. Архитектура

Программа разбита на четыре пакета. Главное правило: зависимости идут только сверху вниз, а `model` ничего не знает ни об интерфейсе, ни об анализе. Поэтому правила модели (уникальные коды, вес в [−1; 1], каскадное удаление связей) работают одинаково и при ручном вводе, и при загрузке из файла, и при отмене действий.



Технологии: Swing (интерфейс, граф рисуется вручную через `Graphics2D`, без графовых библиотек), Apache Commons Math (собственные числа), JFreeChart (графики), Gson (JSON), JUnit 5 (185 тестов).

3. Какой класс за что отвечает

model: данные и правила

Класс	Назначение
`Concept`	Вершина: код и название. Пустой код недопустим.
`Edge`	Направленная связь «из, в, вес». Вес в [−1; 1], вес 0 запрещён (у нуля нет знака, а знак нужен для знака цикла).
`FuzzyCognitiveMap`	Сама модель: список вершин и связей. Следит за правилами: уникальные коды, связи только между существующими вершинами, нет дубликатов. `removeConcept()` удаляет вершину вместе со ВСЕМИ её входящими и исходящими связями. `relationMatrix()` строит матрицу W. `nextConceptCode()` выдаёт следующий код C1, C2…

io: файлы

Класс	Назначение
`ModelStorage`	Сохраняет и загружает JSON. При загрузке файл превращается в вызовы `addConcept/addEdge`, поэтому проверки те же, что в редакторе; сообщения об ошибках на русском с номером записи.
`LayoutData`	Положение вершин и изгибы связей, хранятся в файле отдельной необязательной секцией (сама модель их не знает).
`ModelFileException`	Ошибка чтения, записи или проверки файла с готовым для показа текстом.

analysis: вычисления (без интерфейса)

Класс	Назначение
`CycleFinder`	Поиск всех элементарных циклов алгоритмом Джонсона, включая петли; есть лимит числа циклов.
`Cycle`	Один цикл: путь, число отрицательных дуг, произведение весов. Определяет знак цикла.
`StabilityAnalyzer`	Главный класс анализа: считает циклы и собственные числа, содержит оба критерия устойчивости (каждый в одном методе, чтобы их было легко поменять).
`StabilityReport, EigenValue, StabilityStatus`	Результат анализа: циклы, собственные числа (с модулем), статус «устойчива / неустойчива / на границе».
`ImpulseSimulator`	Считает импульсный процесс по формулам слайдов 13–14.
`Scenario, ScenarioPlanner`	Сценарий (в какие вершины и с каким знаком даётся импульс) и составление плана: 3 случайные вершины × {+1, −1} = 6 сценариев.
`ImpulseResult`	Результат одного сценария: значения вершин на каждом шаге; выбирает до 4 вершин для графика.

ui: интерфейс

Класс	Назначение
`MainFrame`	Главное окно: меню, панель инструментов, три вкладки («Редактор», «Анализ», «Импульсы»). Результаты пересчитываются при переходе на вкладку.
`GraphPanel`	Холст редактора: превращает действия мыши в изменения модели (режимы выбор / вершина / связь / удаление), хранит историю отмены, масштаб и сдвиг.
`GraphLayout, EdgeShape`	Где стоят вершины и как изогнуты связи; геометрия связи (кривая и петля), стрелки, попадание мышью.
`GraphPainter`	Только рисует модель: цвета, пунктир, подписи весов, вершины.
`ForceLayout`	Автоматическая расстановка вершин (метод Фрюхтермана и Рейнгольда).
`History`	Отмена и повтор: хранит снимки состояния (до 100 шагов).

`FileController, Dialogs`	Файловые действия и вопрос о несохранённых изменениях; диалоги ввода вершины и веса.
`AnalysisPanel, ReadOnlyTableModel`	Вкладка «Анализ»: таблицы циклов и собственных чисел как на слайде 11.
`ImpulsePanel, ImpulseChartFactory`	Вкладка «Импульсы»: таблица плана эксперимента (как на слайде 12) и графики JFreeChart.
`EditMode`	Режимы редактора и подсказки к ним.

4. Формулы и откуда они взяты

Номера слайдов указаны по материалам лекций: слайды 9, 11, 12 (методика анализа) и 12–17 (импульсные процессы, фото лекции).

4.1. Матрица отношений

> W[i][j] = вес дуги из вершины i в вершину j (0, если дуги нет)

Код: `FuzzyCognitiveMap.relationMatrix()`. **Источник:** слайд 13: вес дуги $x(u_j, u_i)$ из u_j в u_i, матрица смежности A с весами дуг (в программе A = W).

Номера строк и столбцов совпадают с порядком создания вершин.

4.2. Знак цикла и обратная связь

> знак цикла = произведение знаков его дуг
> чётное число отрицательных дуг (или ни одной) → «+», положительная ОС
> нечётное → «−», отрицательная ОС

Код: `Cycle.isPositive()`. **Источник:** слайды 9 и 11. Колонка «Показатель» в таблице циклов = произведение весов дуг цикла (`Cycle.weightProduct()`); это наше предположение, подпись на слайде была обрезана.

4.3. Поиск всех циклов: алгоритм Джонсона

Вершины нумеруются 0…n−1. Для каждой стартовой вершины s ищутся циклы через s по вершинам с номерами не меньше s, поэтому каждый цикл находится ровно один раз и записывается со своей наименьшей вершины. Поиск в глубину ускорен двумя приёмами:

- работаем только внутри компоненты сильной связности вершины s (в неё входят вершины, достижимые из s и из которых можно вернуться в s);
- блокировка вершин: если из вершины уже не удалось вернуться в s, повторно в неё не заходим, пока в пути не появится новая вершина, дающая шанс вернуться.

Петля сразу даёт цикл из одной дуги. Число циклов может быть огромным, поэтому поиск останавливается на лимите 10 000, и интерфейс предупреждает об этом.

Код: `CycleFinder`. **Источник:** D. B. Johnson, «Finding all the elementary circuits of a directed graph», SIAM Journal on Computing, 4(1), 1975. Результат проверен тестом на 200 случайных графах: он совпадает с независимым перебором.

4.4. Структурная устойчивость

> модель структурно устойчива ⇔ число циклов с отрицательной обратной связью нечётно

Код: `StabilityAnalyzer.isStructurallyStable()` (критерий вынесен в один метод). **Источник:** слайд 9.

Следствие критерия: у модели без циклов отрицательных циклов ноль, а ноль чётное число, поэтому структурной устойчивости у неё нет.

4.5. Устойчивость по возмущению и по начальному значению

> $M = \max_i |\lambda_i(W)|, \quad |\lambda| = \sqrt{(Re^2 + Im^2)}$
> $|M| < 1$ — устойчива; $|M| > 1$ — неустойчива; $|M| = 1$ (с допуском 10^{-9}) — на границе устойчивости

Код: `StabilityAnalyzer.analyze()`, `classify()`, `EigenValue.modulus()`. Собственные числа вычисляет `EigenDecomposition` из Apache Commons Math. **Источник:** слайд 11.

Матрица W несимметричная, поэтому собственные числа могут быть комплексными (пары «a ± bi»), и сравнивается именно модуль. Случай $|M| = 1$ на слайде не описан, но встречается в примере, поэтому он выделен в отдельный статус. Смысл критерия: импульс на шаге t равен $P(t) = (W^T)^t P(0)$, и он затухает, только когда все $|\lambda| < 1$.

4.6. Импульсный процесс

> $V(0) = V(\text{исх}) + P(0)$
> $P(t) = W^T \cdot P(t-1)$
> $V(t) = V(t-1) + P(t)$

Это шаговая запись; свёрнутый вид (формула (4) на слайде 14): $V(t) = V(\text{исх}) + (I + W + W^2 + \ldots + W^t)^T P(0)$.

Код: `ImpulseSimulator.simulate()`. **Источник:** слайды 13–16, формулы (1)–(4).

- **P(0)** — вектор начального импульса: ±1 в выбранных вершинах, остальные 0. Импульс разовый, подаётся при t = 0.
- **V(исх) = 0**: начальные значения вершин нулевые (слайд 15), поэтому V(t) — это изменение относительно спокойного состояния.
- **Транспонирование.** Вершина i получает импульс от каждой вершины j по дуге j→i: $P_i(t) = \Sigma_j W[j][i] \cdot P_j(t-1)$. Это и есть формула (1) слайда 13: значение вершины растёт на «вес дуги × изменение источника».

Пример слайда 16: дуги V1→V3 (+1), V1→V2 (+2), V3→V4 (+2), V2→V4 (−1), V4→V1 (+1), импульс P(0) = (1,0,0,0). Тогда P(1) = (0,2,1,0) и V(1) = (1,2,1,0), как на слайде.

> На слайде 16 написано P(2) = (0,0,0,1), но по формулам и весам того же слайда получается P(2) = (0,0,0,0): $p_4 = (-1)\cdot 2 + 2\cdot 1 = 0$. Мы считаем это опечаткой на слайде; тест программы проверяет совпадение с P(1) и V(1).

4.7. План эксперимента и графики

- **Сценарий** — набор возмущений $q_i = \pm 1$ в выбранных вершинах (таблица «сценарий / возмущение / вершины», слайд 12). Положительный (+1) — оптимистический, отрицательный (−1) — пессимистический.
- **План:** 3 случайные вершины × {+1, −1} = 6 сценариев = 6 графиков (`ScenarioPlanner`).
- **График** (слайд 12): ось X — шаги t, ось Y — изменение значений вершин в процентах, на графике до 4 вершин: вершина с импульсом и те, что изменились сильнее всех (`ImpulseResult.chartVertices()`). Проценты: Y = 100 · V(t), единичный импульс принят за 100% (наше допущение; флажок «В процентах» можно выключить).
- **Зеркальность.** Модель линейная, значит $V_-(t) = -V_+(t)$: графики отрицательных сценариев — точные зеркала положительных.

4.8. Формулы интерфейса (кратко)

- **Кривая связи** — квадратичная кривая Безье: $B(t) = (1-t)^2P_0 + 2t(1-t)P_1 + t^2P_2$. Опорная точка P_1 лежит на серединном перпендикуляре к отрезку между центрами вершин на расстоянии «изгиб». Кривая обрезается по границам кругов вершин (поиск параметра t делением пополам), поэтому середина кривой лежит ровно на половине изгиба и при перетаскивании следует за мышью (`EdgeShape.Curve`).
- **Петля** — окружность, пересекающая окружность вершины; точка стрелки — точка пересечения двух окружностей: $a = (R^2 - r^2 + d^2)/(2d)$, $h = \sqrt{(R^2 - a^2)}$ (`EdgeShape.Loop`).
- **Автоматическая расстановка** — силы по Фрюхтерману и Рейнгольду: отталкивание всех пар k^2/d, притяжение вдоль связей d^2/k, где $k = \sqrt{}$(площадь / число вершин); сдвиг ограничен «температурой», которая постепенно падает до нуля (`ForceLayout`). Источник: T. Fruchterman, E. Reingold, «Graph drawing by force-directed placement», Software: Practice and Experience, 1991.
- **Масштаб:** экран = координата · scale + сдвиг; при колёсике точка под курсором остаётся на месте.

5. Вопросы, которые могут задать

Почему матрица транспонируется в импульсном процессе?

W[i][j] — дуга из i в j, а вершина получает импульс от тех, кто на неё указывает: $P_i = \Sigma_j W[j][i] \cdot P_j$. Это умножение на W^T. Тест это проверяет: импульс идёт только вдоль направления дуги.

Почему графики отрицательных сценариев зеркальны?

Процесс линейный: смена знака импульса меняет знаки всех откликов. Это следствие формул, а не ошибка.

Как связаны собственные числа и импульсы?

$P(t) = (W^T)^t P(0)$. Если все $|\lambda| < 1$, импульсы затухают и график выходит на ровный уровень; если есть $|\lambda| > 1$, значения растут без остановки.

Почему отдельный статус «на границе устойчивости»?

Слайд описывает только $|M| < 1$ и $|M| > 1$, а $|M| = 1$ встречается в примерах. Из-за погрешности вычислений равенство проверяется с допуском 10^{-9}.

Что если циклов очень много?

В полном графе на 25 вершинах их астрономическое число. Поиск останавливается на 10 000 циклов, а интерфейс показывает предупреждение, что вывод о структурной устойчивости может быть неточным.

Как удаляется вершина?

Логика в модели (`removeConcept()`), а не в интерфейсе: вместе с вершиной удаляются все входящие и исходящие связи и петля, поэтому «висящих» связей не бывает. Это покрыто тестами.

Почему вес связи 0 запрещён?

Знак цикла считается как произведение знаков дуг, а у нуля знака нет. Связь без влияния просто не создаётся. Проверка помечена в коде маркером «ЗАПРЕТ НУЛЕВОГО ВЕСА».

Как программа проверена?

185 автоматических тестов: правила модели, файл, циклы (сверка с независимым перебором), устойчивость, импульсы, геометрия и поведение редактора через синтетические события мыши.

6. Допущения и открытые места

- «Показатель» в таблице циклов = произведение весов дуг цикла (на слайде подпись обрезана).
- Единичный импульс = 100% по оси Y; на примере слайда 17 по оси просто числа без знака «%».
- V(0) уже включает импульс (формула V(0) = V(исх) + P(0), слайды 13–16); на графике слайда 17 кривые при t = 0 стоят в нуле, то есть там может быть сдвиг на один шаг.
- Расхождение слайда 16 в P(2), см. 4.6.
- Модель без циклов по критерию слайда 9 не считается структурно устойчивой (ноль чётный).
- Лимит циклов (10 000) и число шагов графика (по умолчанию 20, до 200) — настройки реализации, а не часть методики.